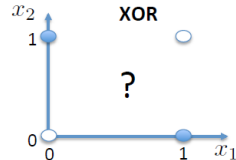
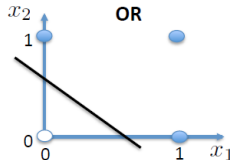
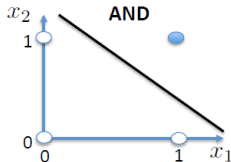


Réseaux de neurones

Daniel Aloise <daniel.aloise@polymtl.ca>
(d'après les slides de H. Larochelle, Google)

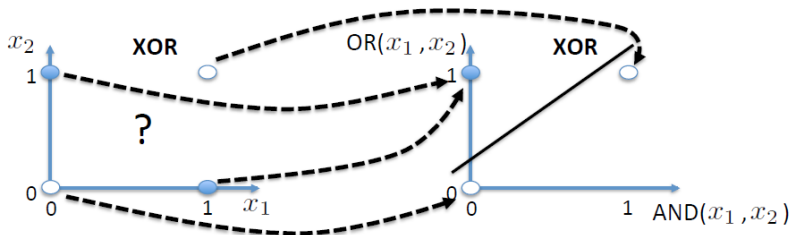
Limitations de classificateurs linéaires

- Le perceptron et la régression logistique ne peuvent pas classer des données non linéairement séparables
- Pourtant, la majorité des problèmes de classification sont non-linéaires



Limitations de classificateurs linéaires

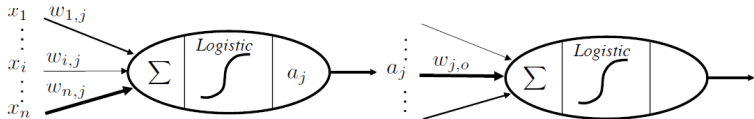
- On peut transformer l'entrée de façon à rendre le problème linéairement séparable sous cette nouvelle représentation
- Dans le cas de XOR :
 - ★ y_1 par $AND(x_1, x_2)$
 - ★ y_2 par $OR(x_1, x_2)$



Réseaux de neurones

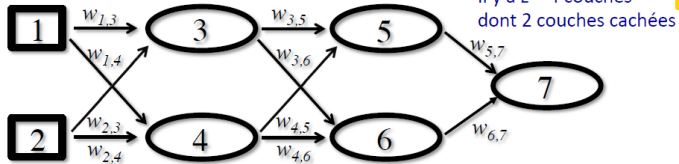
Idée Apprendre les poids du classificateur linéaire **et** une transformation qui va rendre le problème linéairement séparable

- Exemple de réseau de neurones avec une seule **couche cachée**



Cas général

- Rien n'empêche d'avoir plus d'une couche cachée



- On note a_j l'output du j -ème "neurone", incluant les neurones d'entrée et de sortie.
 - ★ Donc on a $a_i = x_i$ pour la couche d'entrée
 - ★ a_N va correspondre à la valeur de la prédiction
- On note aussi in_j l'input du j -ème neurone avant la régression logistique, c.a.d.,

$$\phi(in_j) = \phi \left(\sum_i w_{ij} a_i \right)$$

Dérivation de la règle d'apprentissage

- La dérivation de la règle d'apprentissage se fait avec les gradients

$$w_{ij} \leftarrow w_{ij} - \eta \frac{\partial}{\partial w_{ij}} J(\mathbf{w}, t) \quad \forall i, j$$

- ★ basée sur le gradient **stochastique** avec un seul exemplaire
- ★ la fonction perte J n'est plus convexe



- Calculer ces dérivées partielles peut paraître ardu
- Le calcul est grandement facilité en utilisant la règle de **dérivation en chaîne**

Dérivation en chaîne

- Si on peut écrire une fonction $f(x)$ à partir d'un résultat intermédiaire $g(x)$

$$\frac{\partial f(x)}{\partial x} = \frac{\partial f(x)}{\partial g(x)} \frac{\partial g(x)}{\partial x}$$

- Si on peut écrire une fonction $f(x)$ à partir de résultats intermédiaires $g_i(x)$, alors on peut écrire la dérivée partielle

$$\frac{\partial f(x)}{\partial x} = \sum_i \frac{\partial f(x)}{\partial g_i(x)} \frac{\partial g_i(x)}{\partial x}$$



Dérivation de la règle d'apprentissage

- La règle d'apprentissage est donnée par :

$$w_{ij} \leftarrow w_{ij} - \eta \frac{\partial}{\partial w_{ij}} J(\mathbf{w}, t) \quad \forall i, j$$

- Par l'application de la dérivée en chaîne, on a :

$$w_{ij} \leftarrow w_{ij} - \eta \frac{\partial}{\partial in_j} J(\mathbf{w}, t) \frac{\partial}{\partial w_{ij}} in_j$$

gradient de l'erreur associée à $\mathbf{x}^{(t)}$ p/r à la somme des entrées
au neurone j



Dérivation de la règle d'apprentissage

- La règle d'apprentissage est donnée par :

$$w_{ij} \leftarrow w_{ij} - \eta \frac{\partial}{\partial w_{ij}} J(\mathbf{w}, t) \quad \forall i, j$$

- Par l'application de la dérivée en chaîne, on a :

$$w_{ij} \leftarrow w_{ij} - \eta \frac{\partial}{\partial w_{ij}} J(\mathbf{w}, t) \frac{\partial}{\partial in_j} in_j$$

gradient de la somme des entrées du neurone j p/r au poids w_{ij}

- un calcul naïf de toutes ces dérivées serait très inefficace



Rétropropagation de gradients



- Utilise le fait que la dérivée pour un neurone à la couche ℓ peut être calculée à partir de la dérivée des neurones connectés à la couche $\ell + 1$

$$\begin{aligned} \frac{\partial J(\mathbf{w}, t)}{\partial in_j} &= \frac{\partial J(\mathbf{w}, t)}{\partial a_j} \frac{\partial a_j}{\partial in_j} \\ &= \left(\sum_k \frac{\partial J(\mathbf{w}, t)}{\partial in_k} \frac{\partial in_k}{\partial a_j} \right) \frac{\partial \phi(in_j)}{\partial in_j} \end{aligned}$$



- ★ rappel : $a_j = \phi(in_j)$
- ★ rappel : ϕ est la fonction sigmoïde
- ★ k itère sur les neurones cachées de la couche $\ell+1$



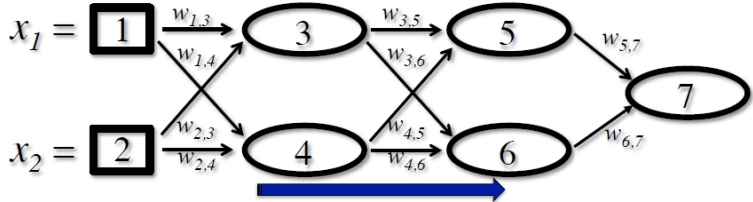
- on continue...

$$\begin{aligned}
 \frac{\partial J(\mathbf{w}, t)}{\partial in_j} &= \frac{\partial J(\mathbf{w}, t)}{\partial a_j} \frac{\partial a_j}{\partial in_j} \\
 &= \left(\sum_k \frac{\partial J(\mathbf{w}, t)}{\partial in_k} \frac{\partial in_k}{\partial a_j} \right) \frac{\partial \phi(in_j)}{\partial in_j} \\
 &= \left(\sum_k \frac{\partial J(\mathbf{w}, t)}{\partial in_k} w_{jk} \right) \phi(in_j)(1 - \phi(in_j))
 \end{aligned}$$

★ rappel : $in_k = \sum_i w_{ik} a_i$

Visualisation de la rétropropagation

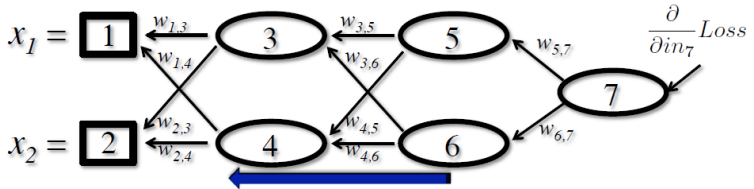
- L'algorithme d'apprentissage commence par une propagation avant



$$a_k = \phi \left(\sum_j w_{jk} a_j \right)$$

Visualisation de la rétropropagation

- Ensuite, le gradient sur la sortie est calculé, et le gradient **rétropropagé**



$$\frac{\partial}{\partial in_j} J(\mathbf{w}, t) = \phi(in_j)(1 - \phi(in_j)) \sum_k w_{jk} \frac{\partial}{\partial in_k} J(\mathbf{w})$$

Retour sur la règle d'apprentissage

$$w_{ij} \leftarrow w_{ij} - \eta \frac{\partial}{\partial in_j} J(\mathbf{w}, t) \frac{\partial}{\partial w_{ij}} in_j$$

- On va simplifier la notation en faisant :
 - $\Delta[j] = -\frac{\partial}{\partial in_j} J(\mathbf{w}, t)$
 - $a_i = \frac{\partial}{\partial w_{ij}} in_j$
- On aura donc :

$$w_{ij} \leftarrow w_{ij} + \eta \Delta[j] a_i$$

Algorithme

Entrée : Un ensemble de données $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots$
Un réseau avec L couches et une fonction
d'activation ϕ
Taux d'apprentissage η

- 1 Pour chaque poids w_{ij} , on fait w_{ij} égal à un petit nombre dans $[-0.01, 0.01]$ choisi au hasard



repeat

 for chaque exemplaire $x^{(t)}$ do

 for chaque noeud i de la couche d'entrée do

$$a_i \leftarrow x_i^{(t)}$$

 end for

 for $\ell = 2, \dots, L$ do

 for chaque noeud j dans la couche ℓ do

$$in_j \leftarrow \sum_i w_{ij} a_i$$

$$a_j \leftarrow \phi(in_j)$$

 end for

 end for

$\Delta[N] \leftarrow -\partial \text{Loss} / \partial in_N$ // N est le neurone de la dernière couche

 for $\ell = L - 1, \dots, 1$ do

 for chaque noeud i dans la couche ℓ do

$$\Delta[i] \leftarrow \phi(in_i)(1 - \phi(in_i)) \sum_j w_{ij} \Delta[j]$$

 end for

 end for

 for chaque poids w_{ij} dans le réseau do

$$w_{ij} \leftarrow w_{ij} + \eta a_i \Delta[j]$$

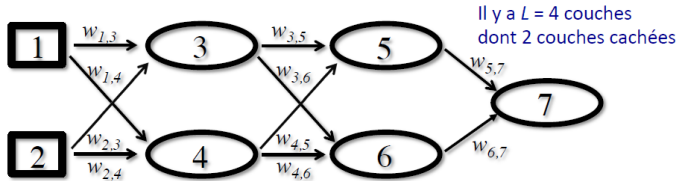
 end for

end for

until critère d'arrêt pas satisfait

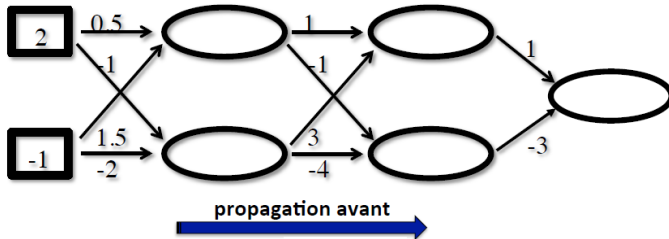
Exemple

- $x = (2, -1), y = 1$



Exemple

- $x = (2, -1), y = 1$

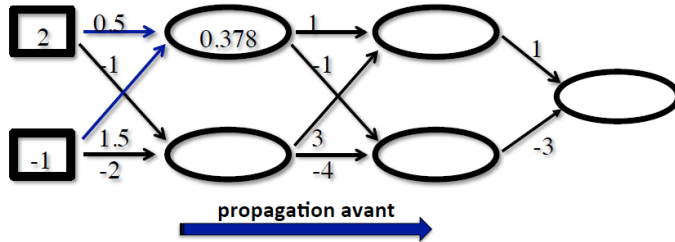


$$a_k = \phi \left(\sum_j w_{jk} a_j \right)$$

ϕ est la fonction sigmoïde

Exemple

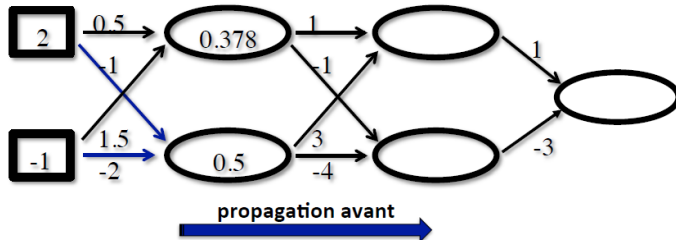
- $x = (2, -1), y = 1$



$$\phi(0.5 \times 2 + 1.5 \times -1) = \phi(-0.5) = 0.378$$

Exemple

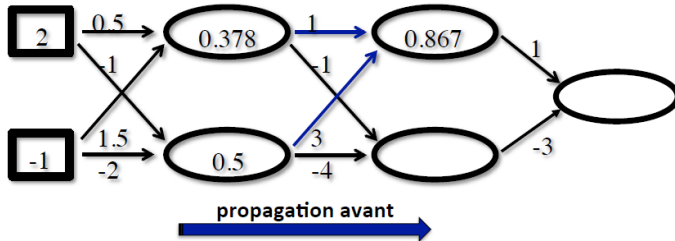
- $\mathbf{x} = (2, -1), y = 1$



$$\phi(-1 \times 2 + -2 \times -1) = \phi(0) = 0.5$$

Exemple

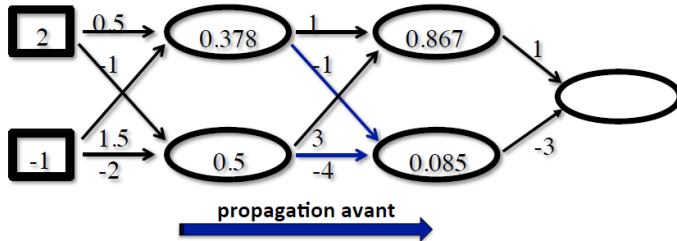
- $x = (2, -1), y = 1$



$$\phi(1 \times 0.378 + 3 \times 0.5) = \phi(1.878) = 0.867$$

Exemple

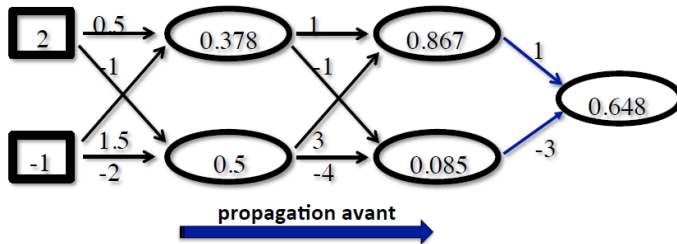
- $x = (2, -1), y = 1$



$$\phi(-1 \times 0.378 + -4 \times 0.5) = \phi(-2.378) = 0.085$$

Exemple

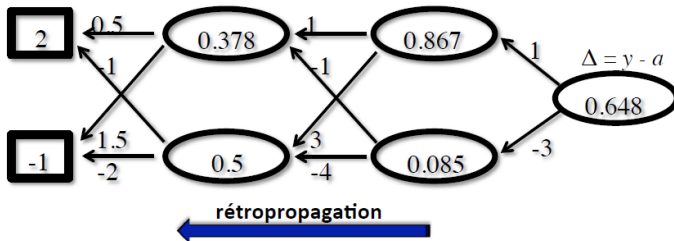
- $x = (2, -1), y = 1$



$$\phi(1 \times 0.867 + -3 \times 0.085) = \phi(0.612) = 0.648$$

Exemple

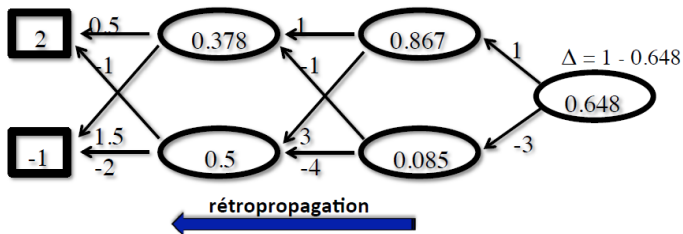
- $x = (2, -1), y = 1$



$$\Delta[j] \leftarrow \phi(in_j)(1 - \phi(in_j)) \sum_k w_{jk} \Delta[k]$$

Exemple

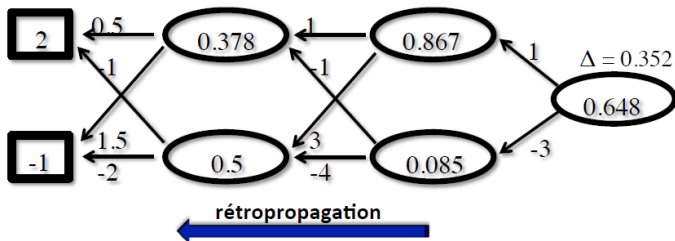
- $\mathbf{x} = (2, -1), y = 1$



$$\Delta[j] \leftarrow \phi(in_j)(1 - \phi(in_j)) \sum_k w_{jk} \Delta[k]$$

Exemple

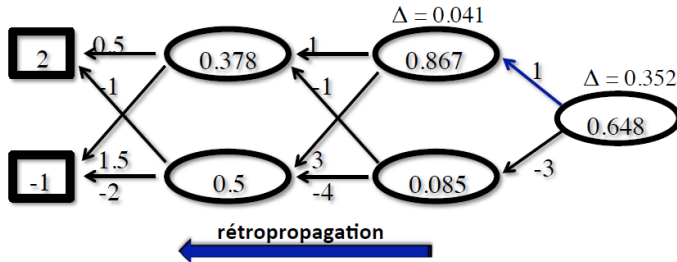
- $x = (2, -1), y = 1$



$$\Delta[j] \leftarrow \phi(in_j)(1 - \phi(in_j)) \sum_k w_{jk} \Delta[k]$$

Exemple

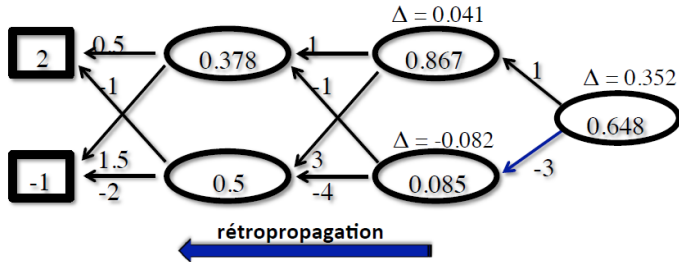
- $\mathbf{x} = (2, -1), y = 1$



$$\Delta = 0.867 \times (1 - 0.867) \times 1 \times 0.352 = 0.041$$

Exemple

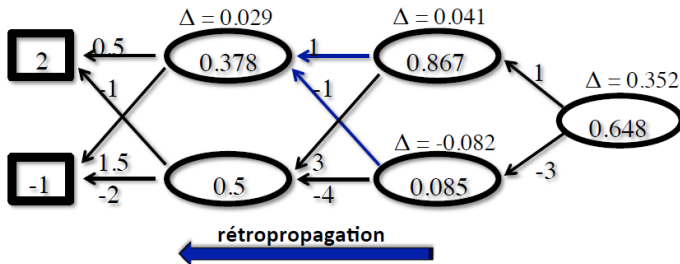
- $\mathbf{x} = (2, -1), y = 1$



$$\Delta = 0.085 \times (1 - 0.085) \times -3 \times 0.352 = -0.082$$

Exemple

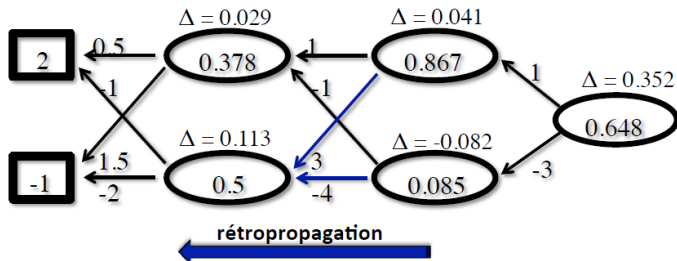
- $\mathbf{x} = (2, -1), y = 1$



$$\Delta = 0.378 \times (1 - 0.378) \times (1 \times 0.041 + -1 \times -0.082) = 0.029$$

Exemple

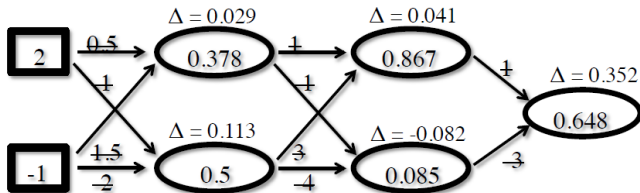
- $\mathbf{x} = (2, -1), y = 1$



$$\Delta = 0.5 \times (1 - 0.5) \times (3 \times 0.041 + -4 \times -0.082) = 0.113$$

Exemple

- $\mathbf{x} = (2, -1), y = 1$
- Mise à jour avec $\eta = 0.1$



$$w_{1,3} \leftarrow 0.5 + 0.1 * 2 * 0.029 = 0.506$$

$$w_{1,4} \leftarrow -1 + 0.1 * 2 * 0.113 = -0.977$$

$$w_{2,3} \leftarrow 1.5 + 0.1 * -1 * 0.029 = 1.497$$

$$w_{2,4} \leftarrow -2 + 0.1 * -1 * 0.113 = -2.011$$

$$w_{3,5} \leftarrow 1 + 0.1 * 0.378 * 0.041 = 1.002$$

$$w_{3,6} \leftarrow -1 + 0.1 * 0.378 * -0.082 = -1.003$$

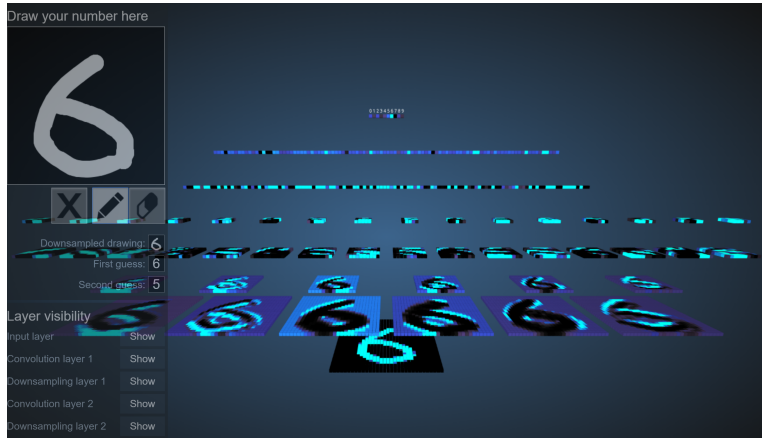
$$w_{4,5} \leftarrow 3 + 0.1 * 0.5 * 0.041 = 3.002$$

$$w_{4,6} \leftarrow -4 + 0.1 * 0.5 * -0.082 = -4.004$$

$$w_{5,7} \leftarrow 1 + 0.1 * 0.867 * 0.352 = 1.031$$

$$w_{6,7} \leftarrow -3 + 0.1 * 0.085 * 0.352 = -2.997$$

For fun

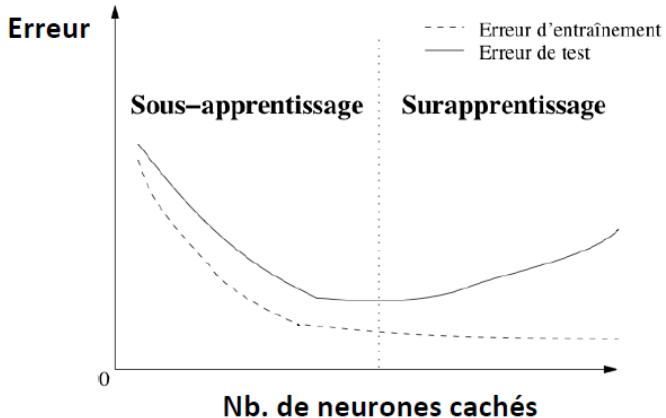


Généralisation

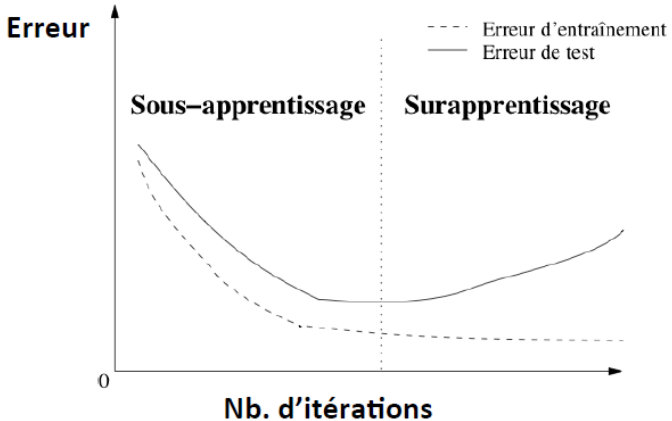


- Comment évaluer le succès d'un algorithme ?
 - On pourrait regarder l'erreur moyenne commise sur les exemples d'entraînement, mais cette erreur est **optimiste**
- Ce qui nous intéresse vraiment, c'est la capacité de l'algorithme à généraliser sur de nouveaux exemples (**ensemble de test**)
- Les algorithmes d'apprentissage ont normalement des **hyper-paramètres** à déterminer
 - perceptron, régression logistique : taux d'apprentissage, nombre d'itérations, etc.
 - réseaux de neurones : on ajoute la fonction d'activation, nombre de couches de neurones cachés, etc.

- Quelle est la relation entre l'erreur d'entraînement et de test ?



- Quelle est la relation entre l'erreur d'entraînement et de test ?



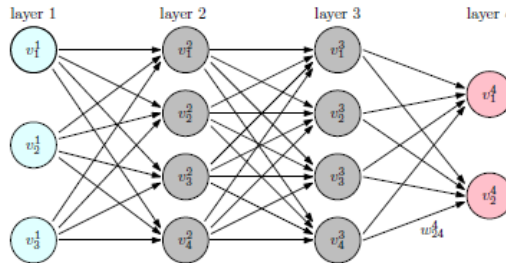
Procédure d'évaluation d'un algorithme d'apprentissage

- Utilisation typique d'un algorithme d'apprentissage supervisé
 - séparer nos données en 3 ensembles : entraînement (70%), validation (15%) et test (15%)
 - faire une liste de valeurs des hyper-paramètres à essayer
 - pour chaque élément de cette liste, lancer l'algorithme d'apprentissage sur l'ensemble d'entraînement et mesurer la performance sur l'ensemble de validation
 - réutiliser la valeur des hyper-paramètres avec la meilleure performance en validation, pour calculer la performance sur l'ensemble de test
- La performance sur l'ensemble de test est alors une estimation non-optimiste de la performance de généralisation de l'algorithme



Apprentissage profond

- Le domaine le plus chaud de l'apprentissage machine implique aujourd'hui de grandes architectures de réseaux neuronaux profonds
- Les **couches cachées** créent de compositions de fonctions non-linéaires $\Rightarrow$ **plus pouvoir de représentation**



Skiena, 2017

Apprentissage profond

- L'apprentissage profond généralement évite le pire comportement du surapprentissage, peut-être en utilisant des moyens moins précis d'encoder la connaissance 

Apprentissage profond

- L'apprentissage profond généralement évite le pire comportement du surapprentissage, peut-être en utilisant des moyens moins précis d'encoder la connaissance 
- L'apprentissage profond est une technologie très excitante, bien qu'il soit mieux adapté aux domaines avec d'énormes quantités de données étiquetées

Apprentissage profond

- L'apprentissage profond généralement évite le pire comportement du surapprentissage, peut-être en utilisant des moyens moins précis d'encoder la connaissance 
- L'apprentissage profond est une technologie très excitante, bien qu'il soit mieux adapté aux domaines avec d'énormes quantités de données étiquetées
- Vous aurez besoin des bonnes GPU pour faire rouler tes expériences

Compte rendu

- À la fin de cette séance, vous avez appris :
 - le fonctionnement et les limitations de classificateurs linéaires
 - Le **truc** des réseaux de neurones pour rendre les données linéairement séparables
 - comment marche l'algorithme de rétropropagation de gradients
 - les principes de base de l'apprentissage profond
- Références
 - Russell & Norvig : section 18.7
 - ★ et, principalement, beaucoup d'autres sources disponibles aujourd'hui sur le web